Synchronized Multi-Line Digital Signal Generation Using Plural Independent Software-Triggered Serializer Peripherals With Open-Loop Fixed Start-Skew Compensation

A Defensive Publication / Technical Disclosure to Establish Prior Art

Author: Gerard Ziemski (Halfmarble LLC) **Published:** Technical Disclosure Commons, Defensive Publications Series — dpubs_series/10442 **Date of first public disclosure:** 2026-06-05 **Version:** 1.0 **Venue:** Technical Disclosure Commons, archived alongside the halfmarble/NEAToBOARD open-hardware project.

> **Purpose of this document.** This is a *defensive publication*. It is released to place the techniques described herein, and reasonable generalizations of them, into the public domain as documented, dated, enabling prior art, so that they remain free for anyone to practice and cannot be removed from the public by a later patent. It is not an assertion of patent rights. The author makes no warranty and grants the permissions stated in the license above. A reference implementation predates this disclosure in the author's private records; this document is its first public disclosure.

Abstract

A method is disclosed for generating two or more *time-aligned* digital output signals — for example a differential pair such as the USB D−/D+ data lines, or more generally any paired, differential, multi-phase, polyphase, quadrature, or multi-channel synchronized output — on a microcontroller or system-on-chip (SoC) that provides **plural independent hardware serializer peripherals** (such as two or more SPI master transmit/MOSI engines) but provides **no single peripheral and no dedicated hardware** capable of driving the desired multi-line signal directly. Each output line is assigned its own serializer and preloaded with a precomputed bitstream. The serializers are started by **back-to-back software trigger writes** (e.g., consecutive stores to each unit's start/command register) that are preceded by CPU pipeline/memory-ordering **fence instructions**, which quiesce the processor so that the unavoidable inter-unit start skew becomes **small, fixed, and deterministic** rather than variable. That now-constant skew is then cancelled **open-loop** by a single **fixed configured output delay** applied to the later-emitting serializer (for example an SPI MOSI output delay expressed in fractions of the bit clock, dummy lead bits, or a clock-phase offset), requiring no runtime measurement or feedback. The result is that independent serializers, though triggered by separate sequential software writes, emit in alignment to within a small fraction of a bit period. A complete reduction to practice is described in which two SPI controllers of an Espressif

ESP32 synthesize a USB full-speed (12 Mbit/s) D–/D+ differential waveform, enabling software USB host signaling on a chip lacking any USB peripheral.

1. Field and Motivation

Many digital interfaces carry information on two or more conductors that must transition in a fixed, tight time relationship. The differential pair is the canonical case: a USB receiver interprets the *difference* between D– and D+, so the two lines must be co-timed to a small fraction of a bit period. Other examples include multi-phase clocking, quadrature signaling, and multi-channel arbitrary waveform or LED-array generation where channels must remain phase-aligned.

A microcontroller lacking dedicated hardware for such a signal (e.g., no USB PHY/serial-interface engine, and no differential serializer) is conventionally forced to "bit-bang" the signal by toggling general-purpose I/O (GPIO) from cycle-counted CPU code. This consumes the CPU, is intolerant of interrupt jitter, and is limited by the per-bit instruction budget — on common parts capping bit-banged USB at *low speed* (1.5 Mbit/s) rather than *full speed* (12 Mbit/s).

A hardware serializer peripheral — for instance an SPI master operated transmit-only (MOSI-only) — is effectively a hardware shift register that clocks a precomputed buffer out of memory at a precise rate with no per-bit CPU involvement, and can therefore sustain higher line rates. But a single such peripheral drives only **one** line, while the target signal needs **two or more co-timed** lines. The natural idea — use one serializer per line — runs into a synchronization problem: the serializers are independent units, each started by its own trigger, and software can only trigger them in sequence, so one unit begins shifting one or more instruction-times before the others. At high bit rates this inter-unit *start skew* is a significant fraction of a bit period and corrupts the paired signal. This disclosure solves that problem.

2. The Technique

The method comprises the following elements; elements (a)–(c) are the core combination, and (d)–(g) are advantageous additions.

(a) Per-line independent serializers. Each of $N \geq 2$ output lines is assigned to its own independently-triggerable hardware serializer peripheral, each preloaded with the precomputed bitstream for its line.

(b) Deterministic-skew start via a CPU fence. The serializers are triggered by writing their respective start/trigger control values in **immediate succession** (e.g., consecutive stores to each unit's command register). Immediately *before* these writes, one or more CPU pipeline/memory-ordering **fence/synchronization operations** are executed to bring the processor to a quiescent state, so that the trigger writes issue with a **minimal and deterministic** inter-unit skew (on the order of a single store instruction) rather than a variable one influenced by prior in-flight memory or exception activity.

(c) Open-loop fixed compensating delay. The later-emitting serializer(s) are configured with a **fixed, constant output delay** sized to cancel the deterministic skew from (b), so that all lines emit

their first and subsequent bits in alignment. Because (b) makes the skew constant, a single fixed delay value suffices; **no runtime feedback, measurement, or closed-loop adjustment is required.**

(d) Idle-level inversion with software pre-inversion. Where the protocol idle level differs from the serializer's idle level, one serializer's output path is inverted (e.g., via the I/O routing matrix or pin polarity) to rest at the protocol idle level, and that line's data buffer is pre-inverted in software so the on-wire data remains correct.

(e) Pre-encoded paired buffers. The protocol physical layer (e.g., line coding such as NRZI, run-length/bit-stuffing, framing, CRC) is computed ahead of time into per-line buffers expressed as a sequence of per-bit line-state tuples, then split into the per-line serializer buffers; buffers may reside in DMA-capable memory and may be allocated contiguously for cache locality.

(f) Clock targeting by search. Serializer clock prescaler/divider settings are selected by searching the divider space for the combination that most closely approximates a target bit rate.

(g) Execution-environment quiescing. The trigger routine runs pinned to a processor core at elevated priority and, during the timing-critical window, disables interrupts / enters a critical section and resides in fast (non-cache-stalling) memory.

3. Reduction to Practice (Specific Embodiment: ESP32, USB Full Speed)

The following was reduced to practice on an Espressif ESP32 (dual-core Xtensa LX6), which exposes two general-purpose SPI controllers ("HSPI" and "VSPI") and **no USB peripheral**. Each controller is operated transmit-only (MOSI-only), 3-wire, half-duplex master. (Source excerpts implementing the load-bearing steps accompany this document — see §7.)

- **Line/peripheral assignment.** USB **D–** → **GPIO12 via HSPI**; USB **D+** → **GPIO13 via VSPI**. Together they form the USB differential pair.
- **Clock (element f).** A 12 MHz bit rate (USB full speed, ≈83.33 ns/bit) is obtained by iterating the controller prescaler and divider ranges and selecting the pair minimizing error versus 12 MHz; the resulting register value is programmed into both controllers. (`bitbangSPI.c`, `setup_clock`.)
- **Idle inversion (element d).** USB full speed idles with D+ high; an SPI line idles low. The D+ controller's serial output/input signal is routed through the I/O matrix with inversion enabled so the line idles high, and the D+ byte buffer is bitwise-complemented in software so the on-wire data is correct after the hardware inversion. Pin drive strength is set weak so an attached device can overpower the host lines as the protocol requires. (`bitbangSPI.c`, `bitbang_init`; `ubbPackets.c`, `_ubb_usb_add_description`.)
- **Encoding (element e).** For each packet (SOF, SETUP token, DATA/device-request, ACK/NAK), packet bytes are assembled with correct field order, bit-endianness, and CRC5/CRC16; the byte stream is NRZI-encoded with a forced transition stuffed after six consecutive 1s, and an EOP plus idle filler appended. Each symbol is expressed as a D–/D+ level pair and written into the HSPI buffer (D–) and the pre-inverted VSPI buffer (D+), both in DMA-capable memory. (`ubbPackets.c`, `ubb_usb_add_data`; `crc.c`.)

- **Synchronized start (elements b, c).** With both controllers preloaded, the routine executes a fence — on Xtensa: `excw;` then `isync; dsync; esync;` then several `nop`s — immediately followed by back-to-back stores of the "user start" command value to each controller's command register (`spi1->cmd.val = ONE; spi2->cmd.val = ONE;`). The fence makes controller 1 lead controller 2 by a fixed, minimal interval. At initialization, a fixed MOSI output delay (expressed in eighths of the bit-clock period) is configured on the appropriate controller so that this constant skew is cancelled and D–/D+ transition together. An optional busy-spin waits for both units' start bits to self-clear. (`bitbangSPI.c`, `bitbang_write_impl` and the `delay` branch of `bitbang_init`.)
- **Timing isolation (element g).** The transmit routine runs as a highest-priority task pinned to the application core; during transmission it disables interrupts / enters a critical section (logging suppressed) and runs from instruction RAM. Inter-packet timing (e.g., 1 ms keep-alive SOF cadence) is produced by busy-wait.
- **Result.** The two independent controllers jointly emit a valid USB full-speed differential waveform sufficient to drive bus reset, keep-alive, and control transfers (e.g., a SET_ADDRESS enumeration step) on a microcontroller having no USB hardware.

4. Generalizations Disclosed (Defensive Scope)

The following generalizations are expressly disclosed as part of this prior art so that neither the specific technique nor these variations may later be removed from the public domain by patent:

- **Number of lines.** N ≥ 2 lines, each driven by its own serializer, triggered in succession, with a per-unit fixed compensating delay sized to each unit's deterministic skew.
- **Type of serializer.** Any hardware peripheral that shifts a buffer out at a clocked rate and is independently triggerable — SPI master MOSI, I2S/PCM transmitter, UART/USRT, dedicated shift-register block, timer output-compare/PWM-pattern engine, or programmable I/O serializer.
- **Means of making the skew deterministic.** Any pipeline/memory-barrier or instruction-ordering construct appropriate to the architecture (e.g., Xtensa `excw` / `isync` / `dsync` / `esync`, ARM `DSB` / `ISB`, RISC-V `fence`), and/or issuing the triggers from a single uninterruptible window, and/or any single-transaction multi-unit trigger the hardware permits.
- **Means of the compensating delay.** A configured serializer output delay, inserted fixed dummy/lead bits in the earlier-emitting buffer, a clock-phase offset, an earlier clock-enable of the lagging unit, or preamble padding — all **open-loop** (fixed, without runtime feedback).
- **Idle-level establishment.** Inverting one serializer's routed output with software pre-inversion, external inverting buffers, configurable pin polarity, or serializer idle/clock-polarity settings.
- **Target signal / protocol.** USB at low, full, or higher speeds as the processor allows; any other protocol or signal carried on two or more co-timed lines — differential, single-ended paired, multi-phase, polyphase, quadrature, or multi-channel phase-aligned (including multi-channel DAC or LED-array drive).
- **Processor/SoC.** Any single- or multi-core microcontroller or SoC with plural independent serializers. In a multi-core variant the per-line triggers may issue from different cores configured to fire within a bounded window, again with a fixed compensating delay.

- **Encoding.** Any line coding precomputed into the paired buffers as appropriate to the target protocol.

5. Relationship to Prior Art

This section surveys the surrounding public art and states precisely what is, and is not, believed novel, so that the defensive disclosure is accurate and its distinguishing combination is clear. Three categories of prior art are adjacent but distinct.

5.1 Single peripheral driving multiple lines (no inter-unit skew exists). A large body of work drives several output lines from **one** serializer or one parallel-output peripheral, so the synchronization problem addressed here does not arise: ESP32/ESP8266 LED drivers serialize many parallel LED lanes from a **single I2S** peripheral with DMA — e.g., the FastLED ESP32 parallel driver [7], the *I2SClocklessLedDriver* (16 lanes from one I2S) [6], and cnlohr's *esp8266ws2812i2s* (WS2812 from one I2S) [11]; quad-SPI (QSPI) drives four synchronized data lines from a **single** master, e.g., US 9,734,099 B1 [10]. Inter-line alignment in all these is inherent to using one peripheral and is **not** the problem solved here.

5.2 Repurposing one serializer as a bit-stream generator for an unsupported protocol. Vendor application notes emulate one protocol with a single peripheral of another type — e.g., I²S emulated on a **single SPI** with a timer generating the word-select frame: ST AN5086 [4] and NXP AN4944 [12]. These establish the general idea of abusing a serializer as a bitstream generator, but use a **single** unit and therefore present **no inter-unit start-skew** to solve.

5.3 Multi-serializer alignment by hardware clock sync or closed-loop skew control. Where multiple serializers *are* aligned, the art does so by **hardware** means, not by a software-fenced start plus open-loop fixed delay on a commodity MCU:

- **FPGA SERDES:** multiple OSERDES blocks are aligned by a **shared, phase-aligned clock and a common synchronous reset**, optionally with a forwarded-clock training pattern — Xilinx XAPP585 [8] and XAPP704 (16 OSERDES from one clocking module) [9].
- **Cross-chip clock injection:** synchronizing serializers across separate RP2040 chips is done by driving a common oscillator into each chip's clock input and using PIO synchronized interrupts — Weber et al. (2025) [13].
- **Integrated / closed-loop skew compensation:** per-channel programmable skew delays exist **inside integrated multi-channel transmitters/SERDES**, set from on-chip registers/SRAM (US 8,081,706 B2 [1]), as fine analog differential pre-skew (US 2015/0028929 A1 [2]), or **tuned by receiver feedback** to meet an interface spec such as SFI-5 (US 7,467,056 [3]); related receive-side deskew appears in TI TIDUA38 [14]. These are integrated and/or closed-loop, not an open-loop fixed delay applied to a second discrete, software-started serializer.

5.4 What is believed novel — the combination. No located prior art combines all of: (i) **two or more fully independent serializer peripherals** on one MCU/SoC, each driving one line of a co-timed multi-line output; (ii) started by **back-to-back software trigger writes made deterministic**

by a CPU pipeline/memory fence; (iii) with inter-unit start skew cancelled **open-loop by a single fixed configured output delay** (no feedback); and (iv) applied to produce a **differential (or otherwise paired/multi-phase) signal on a part that has no native peripheral for that signal** — reduced to practice as a USB full-speed D–/D+ generator on the ESP32. The contribution is this *combination*; each ingredient individually is known, but their assembly to solve co-timed multi-line output on a commodity MCU without dedicated hardware is not shown in the located art. (This is an absence-of-evidence conclusion over a non-exhaustive public corpus, not a proof of non-existence.)

6. Statement for the Public Record

The author publishes the technique of §2, the embodiment of §3, and the generalizations of §4 as prior art, effective on the date of first public disclosure stated above, to ensure they remain freely practicable by all and to bar their later removal from the public domain by patent.

7. Reference Implementation (Excerpts)

The load-bearing portions of a working reference implementation (from the author's "ubbUSB" Espressif ESP-IDF / PlatformIO project) accompany this disclosure in `reference-excerpts/`, under the Apache License 2.0:

- **`bitbangSPI.c` / `bitbangSPI.h`** — dual independent SPI-controller initialization (`bitbang_init`), the fixed compensating MOSI delay (the `delay` branch), the brute-force clock search (`setup_clock`), and the fenced back-to-back synchronized start (`bitbang_write_impl`).
- **`ubbPackets.c` / `ubbPackets.h`** — the encoder: NRZI, bit-stuffing, EOP/framing, and the splitting of per-bit D–/D+ line-state pairs into the two per-line serializer buffers (`ubb_usb_add_data`, `_ubb_usb_add_description`), plus packet construction.
- **`crc.c` / `crc.h`** — USB CRC5 (token) and CRC16 (data) per the USB specification.

These are excerpts intended to make the disclosure enabling on the core claims; they are not a standalone-buildable project (the full ubbUSB project additionally contains the transmit task, top-level state machine, and build files). The excerpt files retain their original Apache-2.0 headers; this disclosure document is CC-BY-4.0; the surrounding NEAToBOARD hardware project is licensed separately (CERN-OHL-2).

References

[1] US 8,081,706 B2, *Reducing lane-to-lane skew* (per-channel SRAM-set delay in a multi-channel transmitter). https://patents.google.com/patent/US8081706

[2] US 2015/0028929 A1, *Differential pre-skew* (fine analog delay ~0–28 ps). https://patents.google.com/patent/US20150028929A1/en

[3] US 7,467,056, *Per-lane preskew tuned by receiver feedback (SFI-5)*. https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/7467056

[4] STMicroelectronics, AN5086, *I2S protocol emulation on STM32L0 using a standard SPI peripheral*. https://www.st.com/resource/en/application_note/an5086-i2s-protocol-emulation-on-stm32l0-series-microcontrollers-using-a-standard-spi-peripheral-stmicroelectronics.pdf

[5] ESP-IDF SPI Master driver documentation (ESP32 GP-SPI controllers SPI2/SPI3; per-controller start). https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/spi_master.html

[6] hpwit, *I2SClocklessLedDriver* — 16 parallel LED lanes from one ESP32 I2S. https://github.com/hpwit/I2SClocklessLedDriver

[7] FastLED, ESP32 parallel-output driver (single I2S, multiple lanes). https://github.com/FastLED/FastLED/blob/master/src/platforms/esp/32/README.md

[8] Xilinx, XAPP585, *OSERDES output alignment via synchronous reset and forwarded clock*. https://docs.amd.com/api/khub/documents/JlDZcsu8DsMLJ6KXAZfq5Q/content

[9] Xilinx, XAPP704, *16 OSERDES driven from one TX_CLOCKS clocking module*. http://www.altera.co.kr/_xilinx/html/ref/V4AdvantageCD2/content/source/xapp704.pdf

[10] US 9,734,099 B1/B2, *Synchronized multi-line (QSPI) output from a single master*. https://patents.google.com/patent/US9734099B1/en

[11] cnlohr, *esp8266ws2812i2s* — WS2812 LEDs via one ESP8266 I2S + DMA. https://github.com/cnlohr/esp8266ws2812i2s

[12] NXP, AN4944, *Emulating I2S bus on Kinetis-M using SPI plus a timer for word-select*. https://www.nxp.com/docs/en/application-note/AN4944.pdf

[13] Weber et al., *Synchronizing RP2040 serializers via common oscillator into XIN and PIO synchronized interrupts*, IET (2025). https://ietresearch.onlinelibrary.wiley.com/doi/full/10.1049/tje2.70067

[14] Texas Instruments, TIDUA38, *Receive-side (MISO) signal-path delay compensation*. https://www.ti.com/lit/pdf/tidua38

[15] US 2007/0047667 A1; US 8,132,040 — additional inter-channel skew-compensation art. https://patents.google.com/patent/US20070047667A1/en · https://www.freepatentsonline.com/8132040.html

---